The Generated DC-RS Cheatsheet on APEX-Agents (Management Consulting)

REUSABLE CODE AND TOOL-CALL SNIPPETS

To explore and load data from .xlsx files, first use sheets_server_sheets with action "list_tabs" on the file_path to identify worksheets and their indices, then use action "read_tab" with tab_index, max_rows (e.g., 30–50), and compact=true to retrieve structured data without full file load.

```
sheets_server_sheets({"request":{"action":"list_tabs","file_path":"<path_to_xlsx>"}})
sheets_server_sheets({"request":{"action":"read_tab","file_path":"
<path_to_xlsx>","tab_index":0,"max_rows":30,"compact":true}})
```

Use filesystem_server_list_files with recursive path navigation starting from "/" to discover folders and files (e.g., Research, Client Data Room, .xlsx, .docx, .png), then apply filesystem_server_read_image_file for PNG files containing tables or charts.

```
filesystem_server_list_files({"path":"/3. Research/Competitor Pricing and Packaging Research"})
filesystem_server_read_image_file({"file_path":"<path_to_png>"})
```

For multi-step analysis, initialize and update progress with todo_write using a list of objects each having id, status ("in_progress", "pending", "completed"), and content; merge=true on updates to track tasks like data loading, parsing, calculation, and verification.

```
todo_write({"todos":[{"id":"1","status":"in_progress","content":"Discover and load data files"},
{"id":"2","status":"pending","content":"Parse segment/tier data and apply formulas"}],"merge":false})
```

For survey data analysis, load xlsx with pandas, filter by priority criteria (e.g., AI capabilities as top priority), compute mean and standard deviation of efficiency score column for full and filtered datasets.

```
import pandas as pd
df = pd.read_excel('<path_to_survey_xlsx>')
mean_all = df['Efficiency_Score'].mean()
mean_filtered = df[df['Priority'] == 'AI capabilities']['Efficiency_Score'].mean()
std = df['Efficiency_Score'].std()
fraction = abs(mean_all - mean_filtered) / std
```

When pandas or openpyxl are unavailable in code_execution_server_code_exec, fall back to sheets_server_sheets read_tab (with cell_range for large files) to extract per-tier aggregates such as lost CV, retained ARR, and competitor names from WinLoss and Churn logs; parse CSV-like raw_output with csv module if needed.

```
sheets_server_sheets({"request":{"action":"read_tab","file_path":"
<churn_or_winloss_xlsx>","tab_index":0,"cell_range":"A1:P50","compact":true}})
```

SOLUTION STRATEGIES FOR RECURRING TASK SHAPES

For target-setting or forecasting tasks involving segments/tiers (e.g., Upper Mid-Market, Enterprise) and metrics like revenue share, win rate, or multipliers: (1) locate and read transaction or pricing sheets; (2) filter to "Won" deals only for the named segment/sector; (3) compute aggregates like total realized revenue or average deal size; (4) apply target share/multiplier to derive implied volume or pipeline; (5) round per task rules (whole numbers for counts, nearest dollar for currency) before final delivery as plain text message.

```
Filter Won deals in segment X; total_realized = sum(revenue); implied_deals = (target_share * total_realized)
/ avg_expansion_deal_size; required_pipeline = target_ARR / win_rate.
```

For ranking or scoring tasks with multiple criteria (e.g., policy violations, deal types, approval levels): (1) locate the relevant log/data file; (2) aggregate per-entity metrics (counts exceeding thresholds by type); (3) rank and assign scores 1–4 (extreme values get 1 or 4); (4) assign identical scores to ties; (5) apply tiebreaker (e.g., secondary count) if totals match; (6) sum scores, rank entities, and deliver findings as plain-text message after rounding scores to whole numbers.

```
Aggregate per approver: violation_count, negotiation_excess, pilot_excess, cfo_within; rank and score; total
= sum(scores); rank by total, tiebreak by cfo_count.
```

For survey data tasks involving efficiency scores and priority filters: (1) locate and read the survey xlsx file; (2) compute overall mean and std of efficiency; (3) filter to subset where AI capabilities ranked top priority; (4) compute subset mean; (5) calculate |overall_mean - subset_mean| / std as fraction of std dev; (6) round to 4 decimal places before delivery as plain text.

```
mean_all, std, mean_ai = compute_means_and_std(df); fraction = abs(mean_all - mean_ai) / std; round to 4
decimals.
```

For discount analysis tasks on specific tiers from approval logs: (1) locate and read the discount approval logs xlsx; (2) aggregate discount % and policy threshold per named tier (filter rows by tier); (3) compute average discount and average threshold per tier; (4) calculate % variance as (avg_discount - avg_threshold) / avg_threshold; (5) round results to 0.01% and deliver as plain-text message.

```
Per tier: avg_disc = mean(discounts); avg_thresh = mean(thresholds); variance = (avg_disc – avg_thresh) /
avg_thresh * 100; round to 0.01%.
```

For tier-level competitive pressure and severity scoring tasks using WinLoss and churn data: (1) locate churn and winloss xlsx files; (2) aggregate per pricing tier: total CV of lost deals, retained renewal ARR, competitor names in lost deals, original ARR, churn rate; (3) compute competitor loss ratio; (4) apply conditional adjustments (increase churned ARR 15% if lost CV > retained; reduce lost CV 50% if retained > lost); (5) derive severity score, exposure multiplier, and sensitivity factor per tier; (6) identify highest-severity tier, its top competitor (alpha-first on ties), and compute required ratios/factors; deliver as plain-text message after rounding to 2 decimals.

```
Per tier: loss_ratio = lost_CV / total_CV; apply adjustment condition; severity = adj_pressure + (adj_churn *
(adj_lost / orig_ARR)); multiplier = max_severity / adj_pressure; factor = max_severity * (adj_pressure +
adj_churn); rank tiers by severity.
```

For policy breach stress index tasks using discount approval logs and KPI charts: (1) locate the discount approval logs xlsx; (2) filter to deals where Final_Approved_Discount_% > Policy_Threshold_%; (3) compute Policy Breach % = Final_Approved_Discount_% - Policy_Threshold_%; (4) classify severity using the KPI chart; (5) apply risk sensitivity factor; (6) compute revenue exposure per deal; (7) calculate the stress index as average revenue at risk per breaching deal; round to 2 decimal places before delivery as plain-text message.

```
Filter breaching deals; breach_pct = approved – threshold; severity = chart_lookup(breach_pct); exposure =
revenue * sensitivity(severity); index = mean(exposures); round to 2 decimals.
```

For churn/ARR proportion reduction tasks using discount approval logs and customer segmentation data: (1) locate discount approval logs xlsx and customer segmentation xlsx; (2) aggregate per Pricing_Tier: Overall_ARR (from discount) and Churned_ARR (from segmentation); (3) compute current proportion = Churned_ARR / Overall_ARR; (4) for tiers where proportion > 0.5%, required_reduction = Churned_ARR - (0.005 * Overall_ARR); (5) compute avg_ARR_per_deal from discount data; (6) additional_deals = ceil(total_reduction / avg_ARR_per_deal); deliver results as plain-text message after rounding dollars to nearest whole and deals up.

```
Per tier: proportion = churned / overall; if >0.005: reduction = churned – (0.005*overall); deals =
ceil(total_reduction / avg_deal); round dollars, ceil deals.
```

FORMULAS, DEFINITIONS, AND CONVENTIONS

Optimal price = minimum_price * (1 + optimal_multiplier) where optimal_multiplier is the decimal form of the percentage points above minimum (e.g., 0.10 for 10%). Annual revenue per tier/segment = optimal_price * volume. Total forecast revenue in $m = sum(annual_revenue) / 1_000_000, rounded to 1 decimal.

```
revenue_m = (min_price * (1 + mult) * volume) / 1000000
```

Implied deal volume for expansion target = (target_expansion_share * segment_total_realized_revenue_from_Won_deals) / average_size_of_Upper_Mid-Market_Won_expansion_deal. Required pipeline capacity = target_ARR / Target_Win_Rate where target_ARR equals sum of Won deals in the named sector/segment.

```
implied_volume = (share * total_Won_revenue) / avg_deal_size; pipeline = target_ARR / win_rate
```

For multi-criteria ranking/scoring: rank entities by each metric (descending or ascending per rule), assign scores 1–4 with extremes at 1 or 4 and middle ranks 2–3; identical metrics receive identical scores. Tiebreaker on total: use secondary metric (e.g., Director-level count). Round final scores to nearest whole number.

```
rank_by_metric(M); score = 1 for top/bottom, 4 for opposite; total = sum(scores); tiebreak_total by
Director_count.
```

Fraction of standard deviation = |mean_all - mean_filtered| / std_dev where mean_filtered is mean for users with AI priority as top.

```
fraction = |mean_all – mean_ai| / std
```

% variance from policy threshold = ((avg_discount - avg_policy_threshold) / avg_policy_threshold) * 100, rounded to nearest 0.01%. When user count is required, compute midpoint of company size range (e.g., for "50–250M" range use (50 + 250) / 2).

```
variance = ((mean_disc – mean_thresh) / mean_thresh) * 100; midpoint = (low + high) / 2
```

Competitor Loss Ratio = (Total Contract Value of Lost deals / Total Contract Value of all deals). Adjustment rule: If competitor-lost deal value > retained renewal ARR for tier then increase churned ARR by 15%; else reduce competitor-lost contract value by 50%. Severity Score = Adjusted Competitor Pressure + (Adjusted Churn Rate × (Adjusted Competitor Lost Value / Original ARR)). Competitive Exposure Multiplier = Highest Severity Score / Adjusted Competitor Pressure of that tier. Scenario Sensitivity Factor = Highest Severity Score × (Adjusted Competitor Pressure + Adjusted Churn Rate). Round all final numeric outputs to 2 decimal places.

```
loss_ratio = lost_CV / total_CV; if lost_CV > retained_ARR: churned_ARR *= 1.15 else: lost_CV *= 0.5;
severity = adj_pressure + (adj_churn × (adj_lost / orig_ARR)); multiplier = max_severity / tier_pressure;
factor = max_severity × (tier_pressure + tier_churn)
```

Policy Breach % = Final_Approved_Discount_% - Policy_Threshold_%. Policy Breach Stress Index = average revenue at risk across all policy-breaching deals.

```
breach = approved – threshold; index = avg(revenue_at_risk for breaching deals)
```

For churn/ARR proportion reduction: required ARR reduction to reach 0.5% threshold = Churned_ARR - (0.005 * Overall_ARR) per tier (only for tiers where current proportion > 0.5%). Number of additional deals = ceil(total_reduction_across_tiers / avg_ARR_per_deal). Round dollars to nearest whole; round deals upward.

```
reduction = churned – (0.005 * overall); deals = ceil(total_reduction / avg_deal); round(dollars),
ceil(deals)
```

ENVIRONMENT AND SANDBOX FACTS

sheets_server_sheets supports actions "list_tabs" (returns worksheet names and indices) and "read_tab" (returns raw_output as CSV-like text). filesystem_server tools handle directory listing, file metadata, and image reading (base64 PNGs may embed tables). code_execution_server_code_exec executes the "code" value as Python; provide direct statements without shell wrappers.

```
Use list_tabs then read_tab on .xlsx in Client Data Room or Research folders; read_image_file on .png for
multipliers or subscriber data.
```

For large .xlsx files, read_tab may require multiple calls using cell_range parameter (e.g., "A1:L100", "A101:L200") to retrieve data in batches when row limits apply.

```
sheets_server_sheets({"request":{"action":"read_tab","file_path":"
<path>","tab_index":0,"cell_range":"A1:L100","compact":true}})
```

Churn data files are located at /5. Client Data Room/05 Renewal and churn data (logo churn, revenue churn, reasons)/Brightpath_Renewal_Churn_v1.0.xlsx; WinLoss data at /5. Client Data Room/04 Win-loss deal analysis (with reason codes)/Brightpath_WinLoss_Analysis_v1.0.xlsx. Use sheets_server_sheets to read and aggregate per Pricing_Tier columns including ARR, Churn_Risk_Flag, Renewal_Status, Competitor names.

```
sheets_server_sheets({"request":{"action":"read_tab","file_path":"/5. Client Data Room/05 Renewal and churn
data (logo churn, revenue churn,
reasons)/Brightpath_Renewal_Churn_v1.0.xlsx","tab_index":0,"max_rows":100,"compact":true}})
```

Discount approval logs are at /5. Client Data Room/03 Discount rate logs & approval workflow records/Brightpath_Discount_Approval_Logs_v1.0.xlsx; customer segmentation at /5. Client Data Room/08 Customer segmentation data (SMB - Mid-market - Enterprise - Industry)/Brightpath_Customer_Segmentation_v1.0.xlsx. Aggregate per Pricing_Tier from discount (ARR) and segmentation (Churned_ARR, ARR) for proportion calculations.

```
sheets_server_sheets({"request":{"action":"read_tab","file_path":"/5. Client Data Room/03 Discount rate logs
& approval workflow
records/Brightpath_Discount_Approval_Logs_v1.0.xlsx","tab_index":0,"max_rows":100,"compact":true}})
```

PITFALLS, EDGE CASES, AND VERIFICATION CHECKS

code_execution_server_code_exec calls fail with shell syntax errors or "python: not found" if the code string uses shell constructs (e.g., python -c, unescaped quotes, or newlines passed to sh). Provide only raw Python code (use ; for multiple statements if needed) and test simple expressions first.

```
Correct: {"request":{"code":"print(720 * 1.1 * 100000 / 1000000)"}} – avoid wrapping in python -c or shell
syntax.
```

After calculations, verify rounding matches task (nearest whole number for counts/deals, nearest dollar for currency) and that filters are applied only to "Won" deals before aggregating revenue or averages; confirm final output is delivered as plain-text message rather than internal state.

```
confirm value is rounded_to_nearest_whole(deals) and currency_to_nearest_dollar(ARR); deliver as text
message.
```

For survey data, ensure correct filter on priority column before computing means and std; round final fraction and scores to 4 decimal places.

```
confirm filter matches task priority cue; round(fraction, 4).
```

When processing discount logs for tiers, confirm exact tier name matches (e.g., "Business", "Growth") and apply midpoint calculation to company size ranges only when task explicitly requires user count proxy; verify variance uses relative formula before rounding.

```
confirm tier == "Business" or "Growth"; variance = ((avg_disc – avg_thresh) / avg_thresh) * 100; round both
to 0.01.
```

Before computing severity score, explicitly verify the conditional adjustment rule (lost CV vs retained ARR) has been applied correctly per tier; for the highest-severity tier, extract the most frequent competitor from lost deals (break ties by alphabetical order of competitor name); confirm all ratios and factors use the adjusted (post-rule) values and are rounded to exactly 2 decimal places before final message delivery.

```
confirm adjustment applied (if lost_CV > retained: churn ×=1.15); top_competitor = min(frequent_competitors)
on tie; ratio = severity / adj_churn; round(severity, 2)
```

When computing policy breach stress index, ensure only positive breach deals are included; confirm revenue figure is the appropriate one for "at risk" calculation; verify severity classification matches the chart scale before applying sensitivity.

```
confirm breach > 0; use correct revenue column; match chart for severity.
```

For churn/ARR proportion tasks, confirm exact Pricing_Tier name matches between discount approval and segmentation files before aggregating; verify only tiers with proportion > 0.5% receive reduction calculation; confirm avg_ARR_per_deal uses discount approval ARR values; round dollars to nearest whole and deals upward before delivery.

```
confirm tier names match; reduction only if proportion > 0.005; deals = ceil(total_reduction / avg);
round(dollars), ceil(deals)
```